



自定义支持数据绑定的数据集

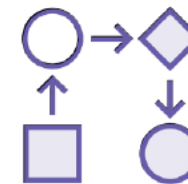
北京理工大学计算机学院
金旭亮

JavaFX控件绑定到自定义数据对象集合



- ✓ 在实际开发中，经常需要在窗体上显示多条记录，这就要求使用诸如 `TableView` 这种能显示多条记录的数据绑定控件，绑定到一个对象集合。
- ✓ 我们希望能实现这样的功能：对象集合中对象数目的增减，集合中单个对象属性的变化，都能够及时地刷新显示。
- ✓ 如果集合中对象的属性是 `JavaFX Bean Property`，底层数据源使用可观察的集合类型，则以上预期目标都可以轻松地达成。

跟踪集合中元素属性值的变化



ObservableList有一个特点，那就是默认情况下它仅监听集合中元素数目的变更，而不监听单个元素中属性值的变更。



如果希望监听单个元素中属性值的变更，则需要添加一个特殊的回调对象，通知ObservableList监听对象的哪些属性值。

下面通过实例说明.....

测试用数据类

//一个属性全部为JavaFX Bean Property的示例数据类

5 usages

```
public class MyJavaFXBeanClass {  
    1 usage  
    public MyJavaFXBeanClass(int num, String info) {  
        this.num.set(num);  
        this.info.set(info);  
    }  
}
```

5 usages

```
private IntegerProperty num = new SimpleIntegerProperty();
```

5 usages

```
private StringProperty info = new SimpleStringProperty();
```

这个类定义了两个JavaFX属性，我们希望当它的info属性值有更改时，它所在ObservableList也会触发“数据已更改”通知。

作为对比，我们有意让另一个num属性值的变化，不会触发“数据已更改”通知。

2 usages

```
public class MyTestHelper {
```

```
    //创建一个测试用数据对象
```

1 usage

```
    public static MyJavaFXBeanClass createExampleObj() {  
        int ranValue = ThreadLocalRandom.current().nextInt();  
        return new MyJavaFXBeanClass(ranValue, info: "info of " + ranValue);  
    }
```

```
    //用于定义哪个属性值的改变需要监听
```

1 usage

```
    public static Callback<MyJavaFXBeanClass, Observable[]> myJavaFXBeanClassExtractor =  
        obj -> new Observable[]{  
            obj.infoProperty()  
        };
```

```
}
```

这里指明要监听MyJavaFXBeanClass对象info属性值的变化

测试代码-1



//跟踪列表以及表中元素属性值的变化

no usages

```
public class MonitorListAndElementProptyChangeTest {
```

no usages

```
public static void main(String[] args) {
```

//创建一个“可观察”的对象集合，并且指定类中的info属性值变化也需要被监听

```
ObservableList<MyJavaFXBeanClass> dataList =
```

```
FXCollections.observableArrayList(MyTestHelper.myJavaFXBeanClassExtractor);
```

//附加改变监听器

```
dataList.addListener((ListChangeListener<MyJavaFXBeanClass>) change ->
```

```
    System.out.println("发生了变化: " + change));
```

```
System.out.println("\n向集合中添加5个元素\n");
```

```
for (int i = 0; i < 5; i++) {
```

```
    dataList.add(MyTestHelper.createExampleObj());
```

```
}
```


输出截图



```
Run  [Icons: Run, Refresh, Stop, Settings, More]
"C:\Program Files\Java\jdk-19\bin\java.exe" ...

↑
↓
≡
≡↓
🖨
🗑

向集合中添加5个元素

发生了变化: { [MyClass{num=IntegerProperty [value: 1274947928], info=StringProperty [value: info of 1274947928]] added at 0 }
发生了变化: { [MyClass{num=IntegerProperty [value: -1303829058], info=StringProperty [value: info of -1303829058]] added at 1 }
发生了变化: { [MyClass{num=IntegerProperty [value: 1311695763], info=StringProperty [value: info of 1311695763]] added at 2 }
发生了变化: { [MyClass{num=IntegerProperty [value: 2069318672], info=StringProperty [value: info of 2069318672]] added at 3 }
发生了变化: { [MyClass{num=IntegerProperty [value: 1937492334], info=StringProperty [value: info of 1937492334]] added at 4 }
```

上述输出表明，元素数目的增减，会触发“数据已更改”通知。

测试代码-2



```
System.out.println("\n现在开始修改第一个元素的info属性\n");
dataList.get(0).setInfo("Modify @" + LocalDateTime.now());
System.out.println("\n现在开始修改第二个元素的num属性\n");
dataList.get(0).setNum(1);
System.out.println("\n现在开始修改第三个元素的info属性\n");
dataList.get(2).setInfo("Modify @" + LocalDateTime.now());
```



可以看到，对任何一个元素的info属性值的修改，均触发了“数据已更改”事件。但对num属性值的修改，则没有这个效果，因为我们没有指定ObservableList去关注这一属性值的变化。

现在开始修改第一个元素的info属性

发生了变化: { updated at range [0, 1) }

现在开始修改第二个元素的num属性

现在开始修改第三个元素的info属性

发生了变化: { updated at range [2, 3) }

Process finished with exit code 0

集合排序



JavaFX提供了一个SortedList，可以对一个ObservableList进行自动排序，也就是说，当ObservableList中发生“数据已更改”事件时，它会自动对它进行一次排序。

下面通过实例说明.....

集合排序-准备

```
no usages
public class SortListTest {
    no usages
    public static void main(String[] args) {
        //创建一个“可观察”的对象集合，并且指定类中的info属性值变化也需要被监听
        ObservableList<MyJavaFXBeanClass> dataList =
            FXCollections.observableArrayList(MyTestHelper.myJavaFXBeanClassExtractor);
        System.out.println("\n向集合中添加3个元素\n");
        for (int i = 0; i < 3; i++) {
            dataList.add(MyTestHelper.createExampleObj());
        }
        //输出原始集合内容
        dataList.forEach(System.out::println);
    }
}
```

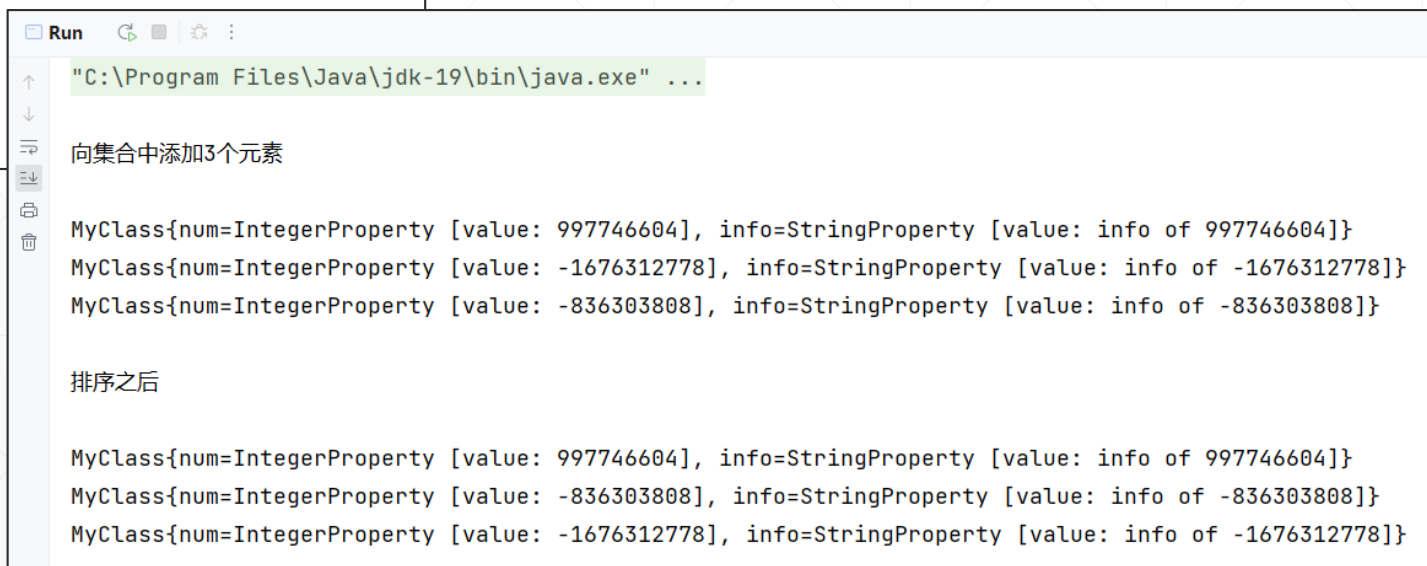
由于info属性值被跟踪，因此，当集合中对象的这个属性值变化时，将会触发“自动排序”功能。

集合排序-指定排序规则

排序是由SortedList完成的，它构造时需要两个参数，一个是包容原始数据的可观察集合，另一个则是指定排序规则：

```
//指定按照info属性值（是一个字符串）的长度进行排序
var sorted = new SortedList<MyJavaFXBeanClass>(dataList, (o1, o2) -> {
    var len1 = Integer.valueOf(o1.getInfo().length());
    var len2 = Integer.valueOf(o2.getInfo().length());
    return len1.compareTo(len2);
});
System.out.println("\n排序之后\n");
sorted.forEach(System.out::println);
```

当遍历SortedList时，得到的就是已经排好序的数据序列。



```
Run
"C:\Program Files\Java\jdk-19\bin\java.exe" ...

向集合中添加3个元素

MyClass{num=IntegerProperty [value: 997746604], info=StringProperty [value: info of 997746604]}
MyClass{num=IntegerProperty [value: -1676312778], info=StringProperty [value: info of -1676312778]}
MyClass{num=IntegerProperty [value: -836303808], info=StringProperty [value: info of -836303808]}

排序之后

MyClass{num=IntegerProperty [value: 997746604], info=StringProperty [value: info of 997746604]}
MyClass{num=IntegerProperty [value: -836303808], info=StringProperty [value: info of -836303808]}
MyClass{num=IntegerProperty [value: -1676312778], info=StringProperty [value: info of -1676312778]}
```

集合排序-修改属性值触发重排序



```
System.out.println("\n修改第一个对象元素的info属性值\n");
System.out.println(dataList.get(0));
dataList.get(0).setInfo("Modified @ " + LocalDateTime.now());
System.out.println("改完了,此对象当前内容为:");
System.out.println(dataList.get(0));
System.out.println("\n再次输出排序集合的内容,看看是否自动排序.....\n");
sorted.forEach(System.out::println);
```

从输出信息中可以看出, info 属性值的改变, 也会触发排序。

修改第一个对象元素的值

```
MyClass{num=IntegerProperty [value: 997746604], info=StringProperty [value: info of 997746604]}
```

改完了,此对象当前内容为:

```
MyClass{num=IntegerProperty [value: 997746604], info=StringProperty [value: Modified @ 17:30:31.357259200]}
```

再次输出排序集合的内容, 看看是否自动排序.....

```
MyClass{num=IntegerProperty [value: -836303808], info=StringProperty [value: info of -836303808]}
```

```
MyClass{num=IntegerProperty [value: -1676312778], info=StringProperty [value: info of -1676312778]}
```

```
MyClass{num=IntegerProperty [value: 997746604], info=StringProperty [value: Modified @ 17:30:31.357259200]}
```

```
Process finished with exit code 0
```